

Formación - Series Temporales

Forecasting - Segunda parte



FACULTAD DE
INGENIERÍA



UNIVERSIDAD
DE LA REPÚBLICA
URUGUAY

Agenda

- Repaso primera parte.
- Forecasting con redes neuronales.
 - Multilayer Perceptron, entrenamiento de una red.
 - Redes recurrentes (RNN).
 - Redes convolucionales (CNN).



Repaso



FACULTAD DE
INGENIERÍA

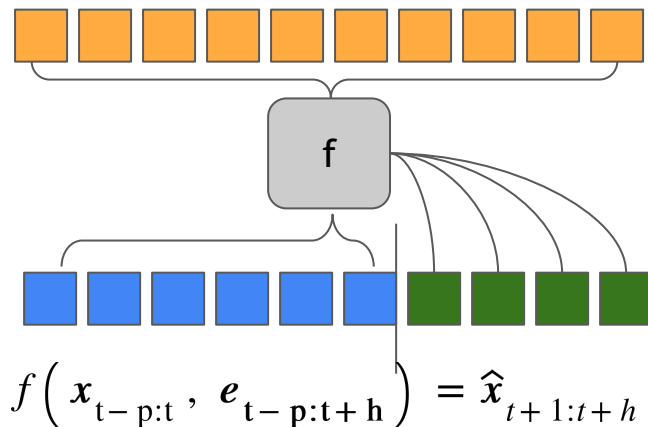
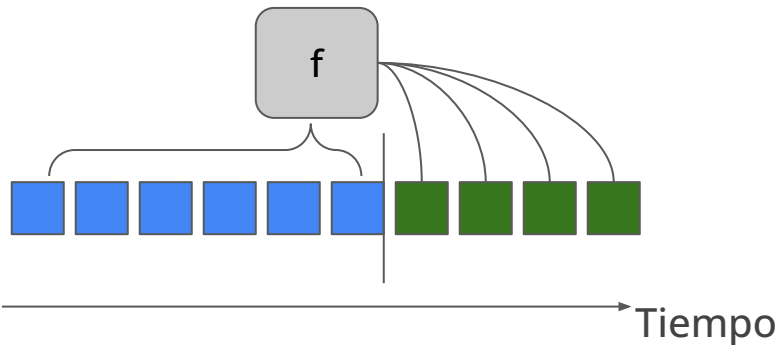


UNIVERSIDAD
DE LA REPÚBLICA
URUGUAY

Introducción - Qué es el forecasting en series temporales?

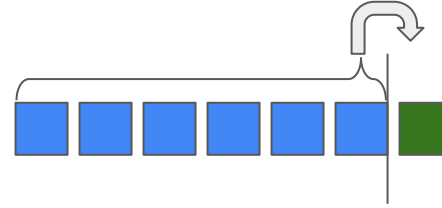
- El forecasting en series temporales (ST) es la tarea de predecir **valores futuros** a partir de sus **valores históricos**.
- Las ST tiene diferentes patrones como la **tendencias**, o la **estacionalidad**; que aportan información que nos ayudan a predecir sus valores futuros.
- Existen diferentes métodos para resolver la tarea: **estadísticos, machine learning, o deep learning**.
- También se pueden utilizar variables **exógenas o covariables**.
- **A más grande el valor de h más compleja es la tarea.**

$$f(x_{t-p:t}) = \hat{x}_{t+1:t+h}$$



Diferentes formas de predecir

Predicción del instante siguiente

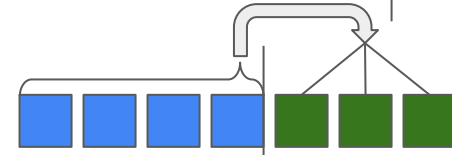


Predicción de una ventana

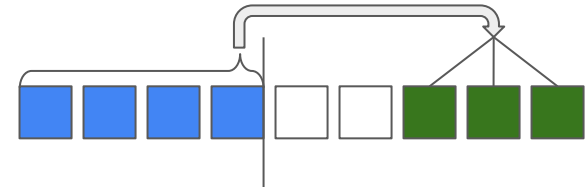
Instante a instante



Completa



Completa espaciada



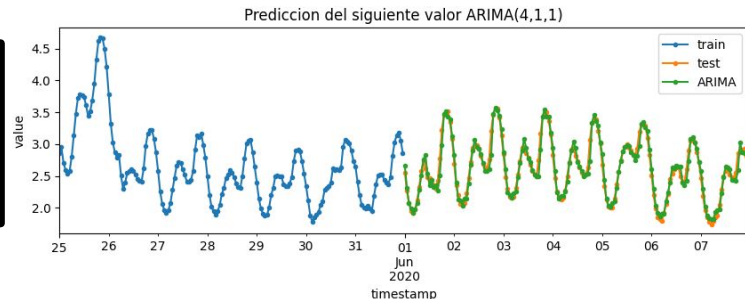
Predicción del instante siguiente: SARIMA

ARIMA

```
arima_model =  
pm.auto_arima(dta_train,  
              seasonal=False)
```

Hiperparámetros: $p=4$, $d=1$, $q=1$

MAE = 0.072
MSE = 0.010
MAPE = 0.028

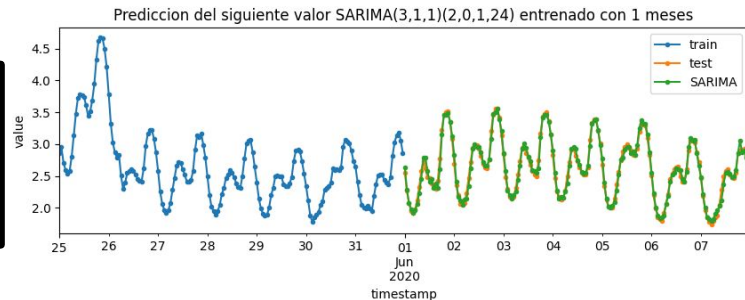


SARIMA

```
sarima_model =  
pm.auto_arima(dta_train,  
              seasonal=True,  
              m=24)
```

**Hiperparámetros: $p=3$, $d=1$, $q=1$,
 $P=2$, $D=0$, $Q=1$**

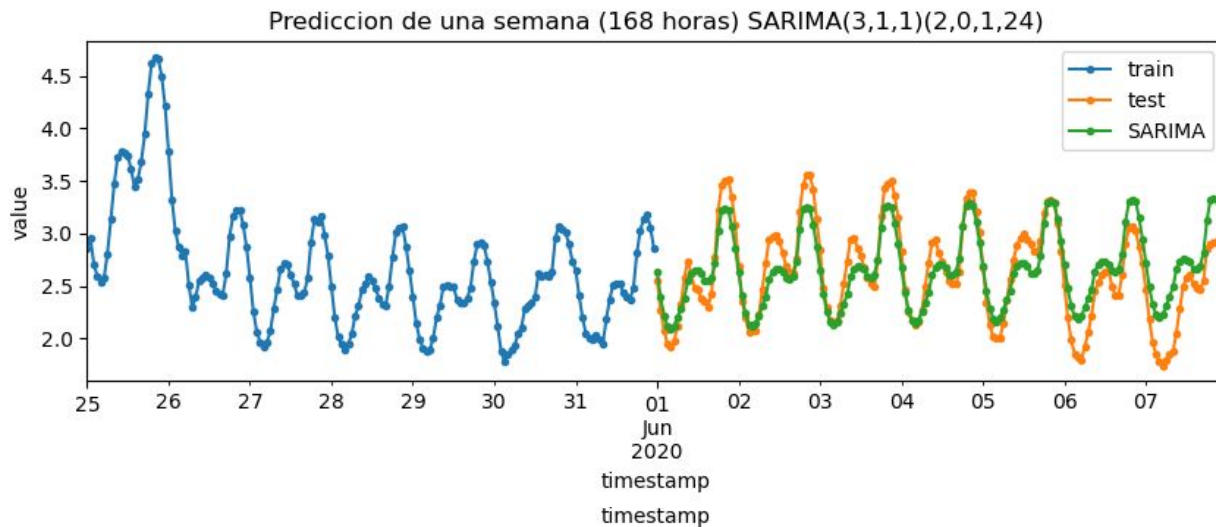
MAE = 0.047
MSE = 0.004
MAPE = 0.019



Predicción de una ventana (1 semana): SARIMA

Predicción de una semana

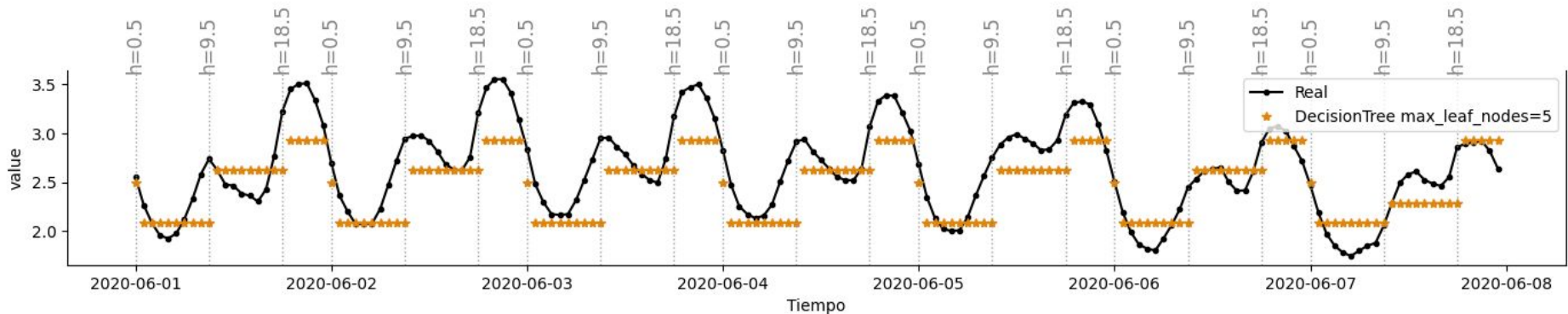
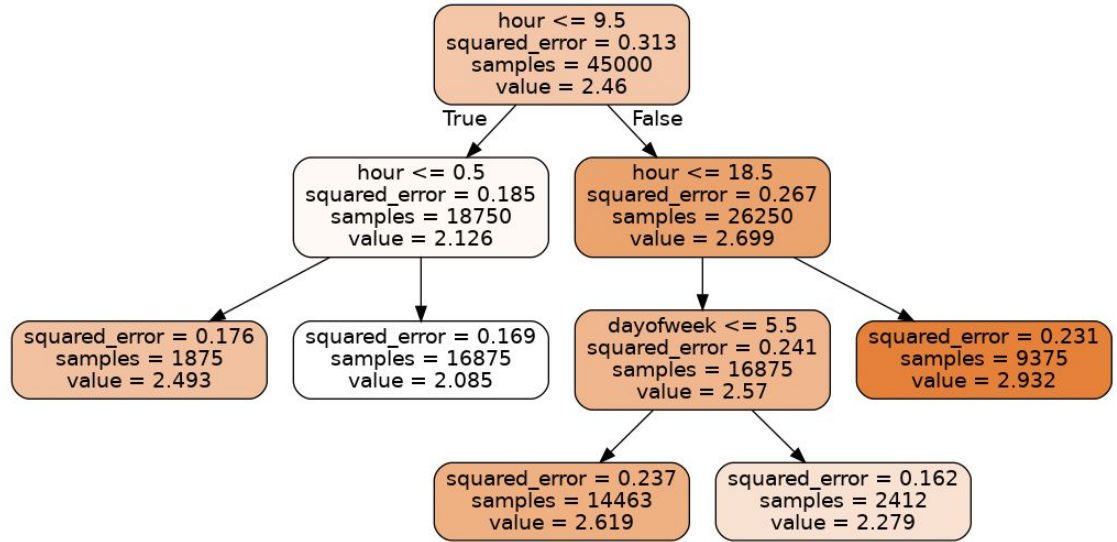
```
sarima_model.forecast(steps=168)
```



Árboles

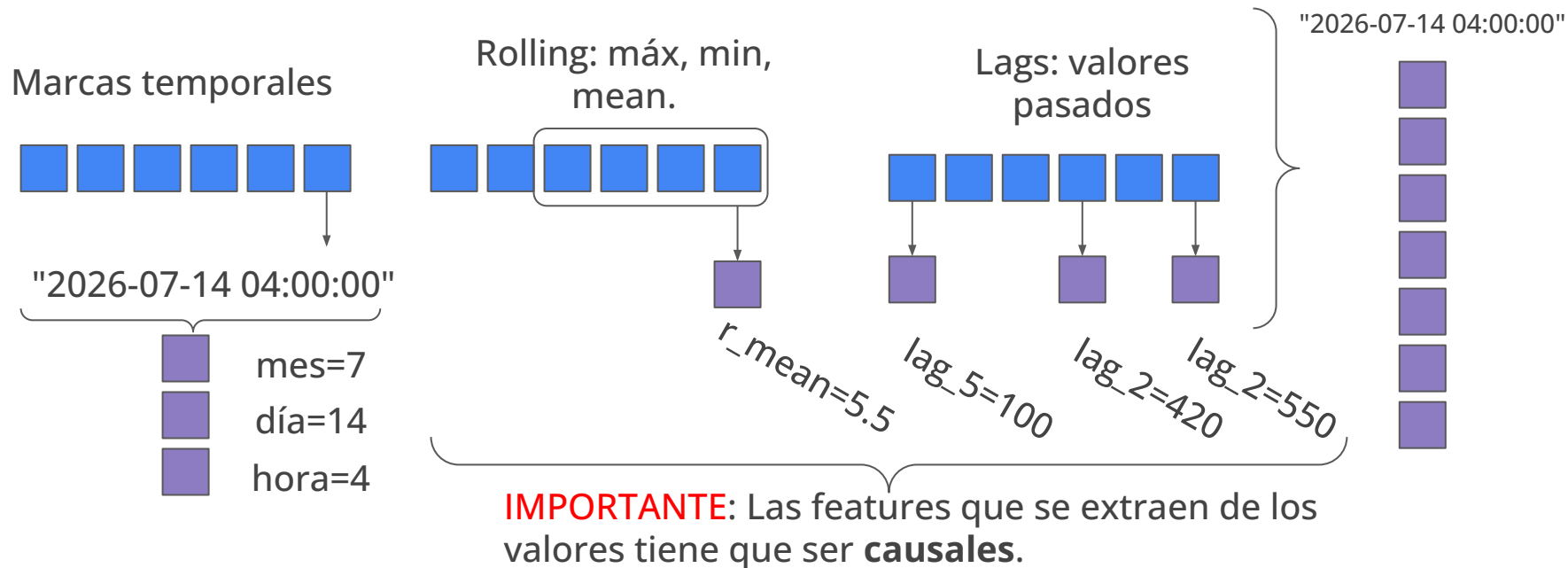
Regresión: Datos de demanda.

- Feature:
 - Hora: [0,1,...,23].
 - Día de la semana: [0,...,6]
 - Día del mes: [0, 30]
- Hiperparámetro:
 - **max_leaf_nodes=5.**



Random Forest

- Mismos hiperparámetros de regularización que los árboles.
- Nuevo hiperparámetro: cantidad de estimadores (árboles).
- Forecasting de series temporales: **regresión de valores futuros**.
- Features para cada instante de tiempo:



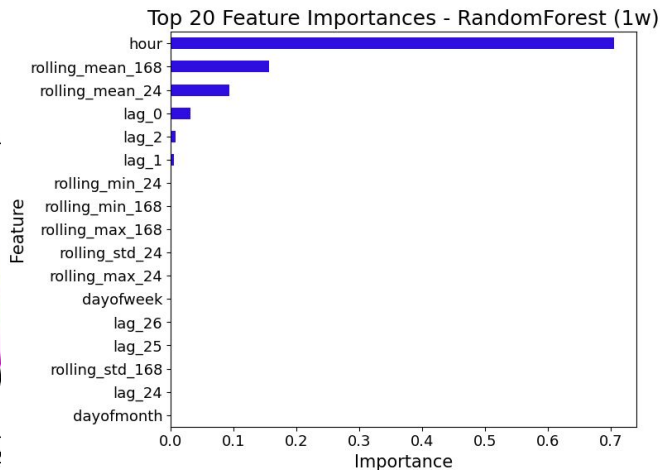
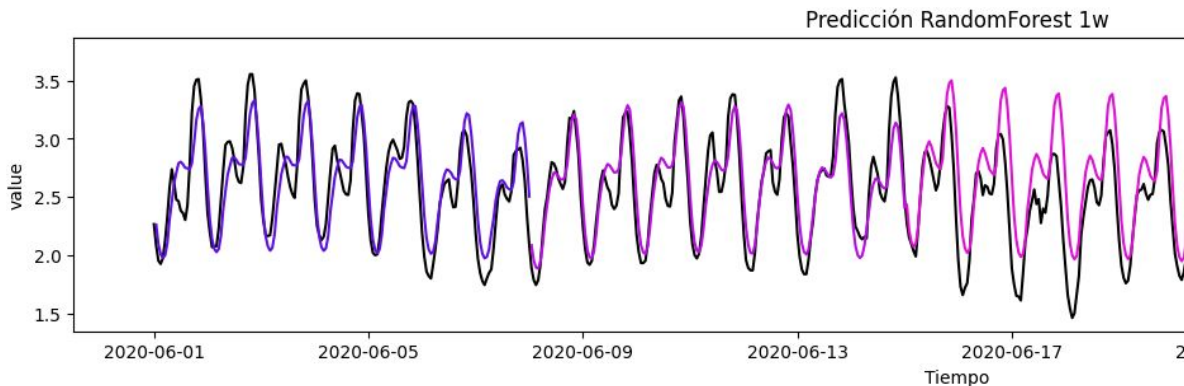
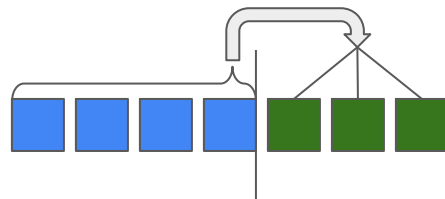
Predicción de una ventana (1 semana): Random Forest

Features:

- Lags: [0, 12].
- Rolling: Ventanas: {24, 168} - mean, std, max, min.
- Marcas: hora, día de la semana, día del mes.

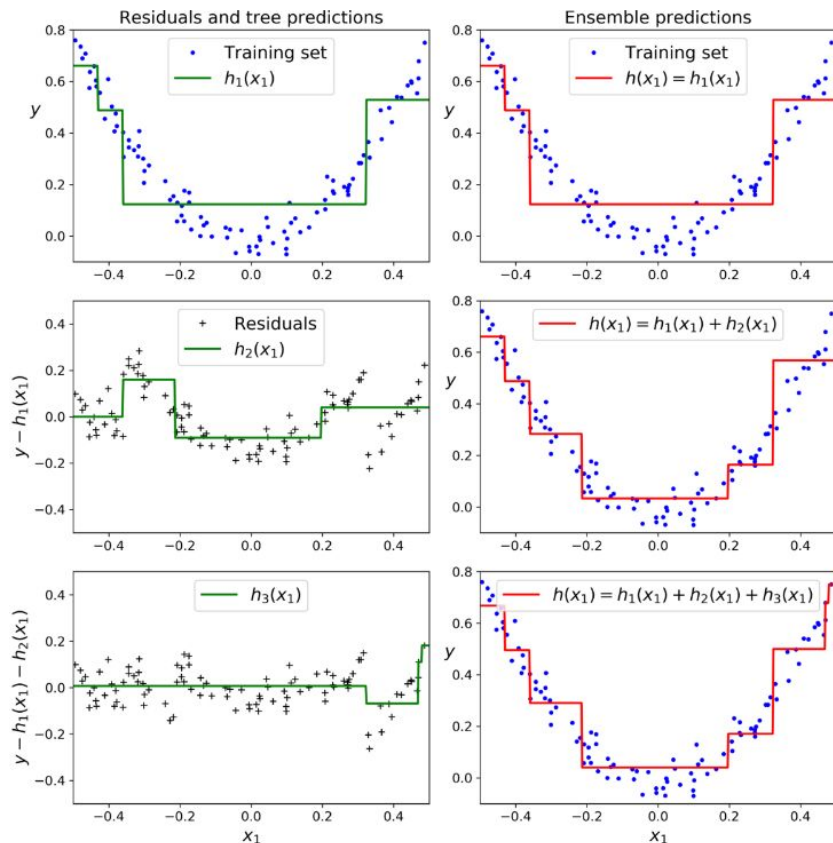
```
rf_1w = RandomForestRegressor(n_estimators=50,  
                              max_leaf_nodes=500)
```

MAE = 0.239
MSE = 0.090
MAPE = 0.090



Boosting: XGBoost

- Concatenación secuencial de árboles.
- A partir del primer árbol, los siguientes se entrenan para hacer un regresión del residuo.
- El entrenamiento secuencial puede ser muy lento.
- Extreme Gradient Boosting (XGBoost): muy rápido, escalable, y portable.

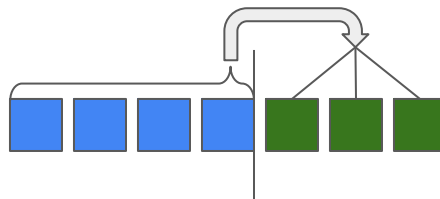


Predicción de una ventana (1 semana): XGBoost

Features:

- Lags: [0, 12].
- Rolling: Ventanas: {24, 168} - mean, std, max, min.
- Marcas: hora, día de la semana, día del mes.

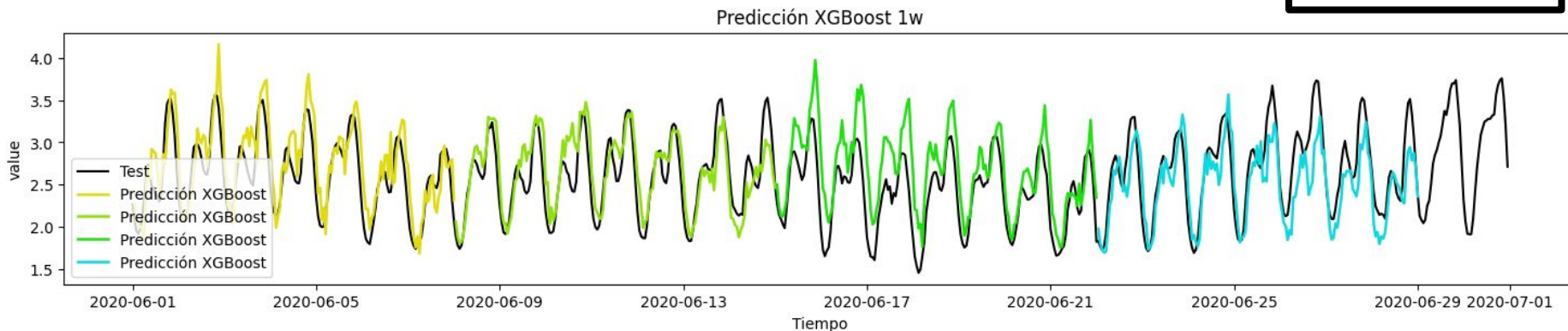
```
xgb_1w = RandomForestRegressor(n_estimators=50,  
                                max_leaf_nodes=500)
```



RF

MAE = 0.239
MSE = 0.090
MAPE = 0.090

MAE = 0.230
MSE = 0.084
MAPE = 0.087



Métodos de forecasting: Redes neuronales



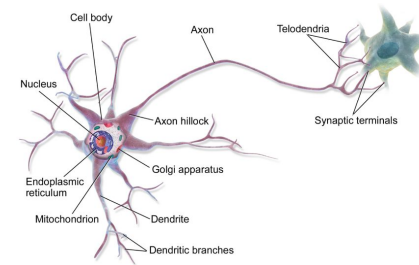
FACULTAD DE
INGENIERÍA



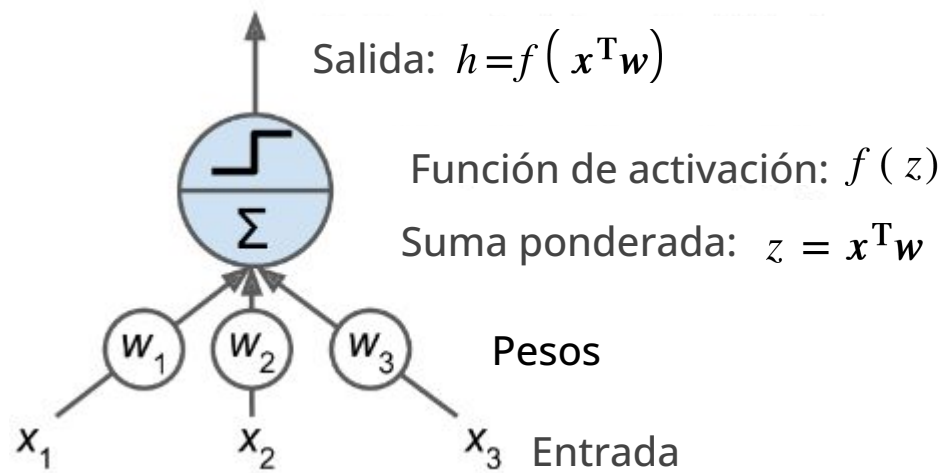
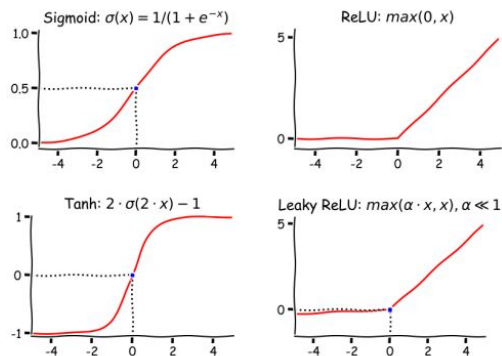
UNIVERSIDAD
DE LA REPÚBLICA
URUGUAY

Perceptron Multicapa

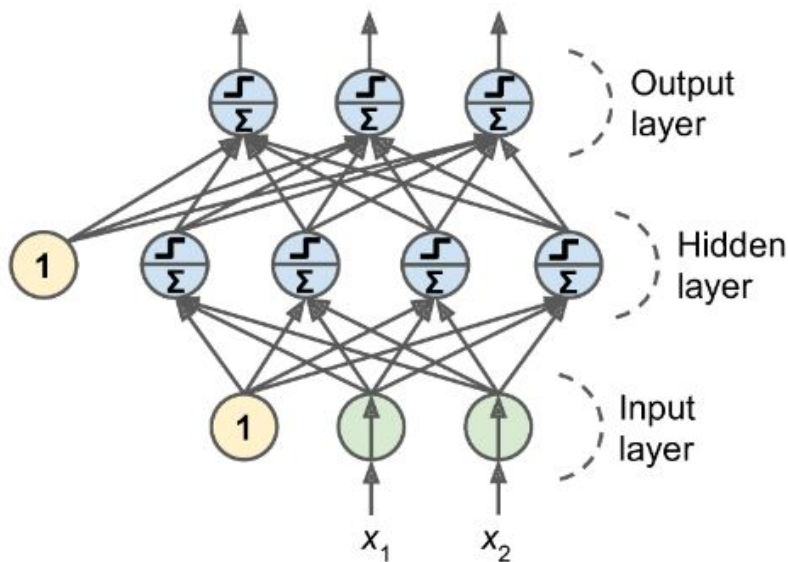
Redes Neuronales: Perceptron o Neurona (unidad)



- Modelo matemático de una neurona.
- Unidad básica en una red neuronal.
- Se compone de:
 - Pesos: parámetros entrenables.
 - Función de activación: no lineal.

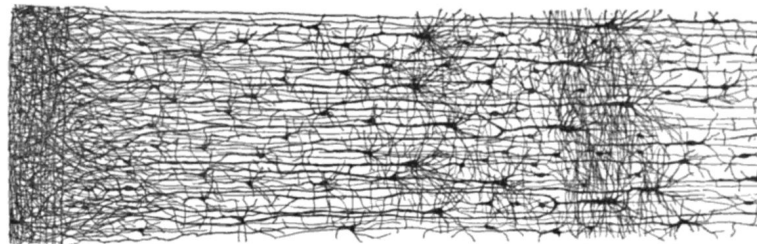


Redes Neuronales: Perceptron multicapa.



Entradas:

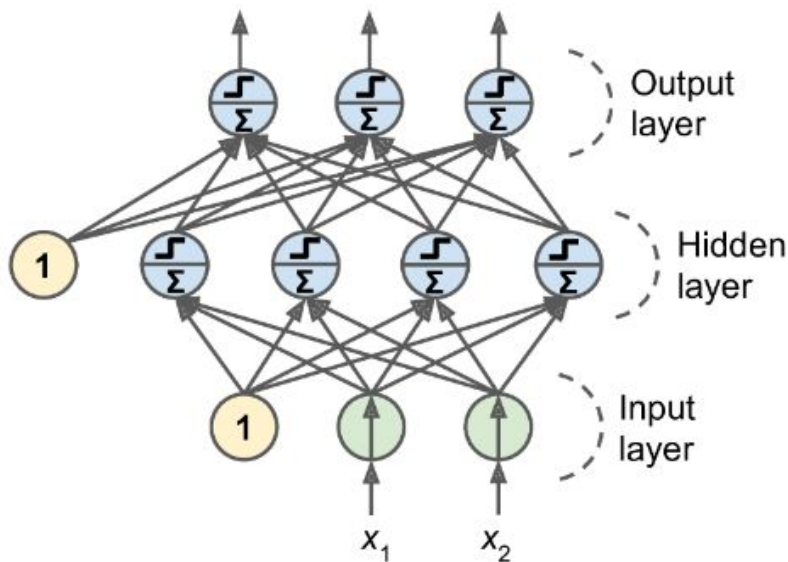
- Crudas. Las features se aprenden.
- Valores estandarizados.



- Red: una o más capas ocultas.
- Hiperparámetros (H):
 - # Capas.
 - # Unidades por capa.
 - Bias: si o no.
 - Función de activación por capa.

Parámetros.

Redes Neuronales: Perceptron multicapa.



Para una entrada de 168 valores (1 semana):
691 parámetros.

Parámetros.

Input layer: 0 parámetros.

$$x_{[2]}$$

Hidden layer (4 unidades): 12 parámetros.

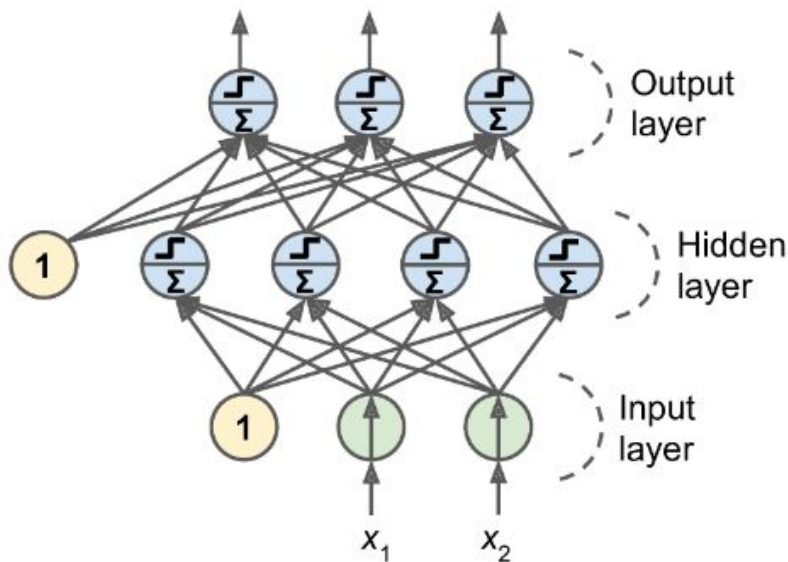
$$f^{(h)}\left(x_{[2]}W^{(h)}_{[2,4]}+b^{(h)}_{[4]}\right)=h_{[4]}$$

Output layer (3 unidades): 15 parámetros.

$$f^{(o)}\left(h_{[4]}W^{(o)}_{[4,3]}+b^{(o)}_{[3]}\right)=o_{[3]}$$

Total: 27 parámetros.

Redes Neuronales: Perceptron multicapa.



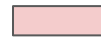
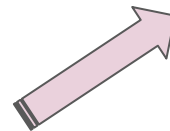
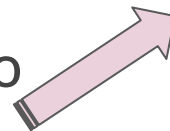
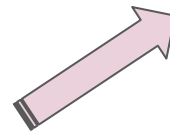
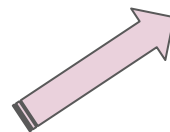
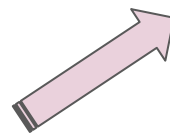
Parámetros.

Capacidad de la red

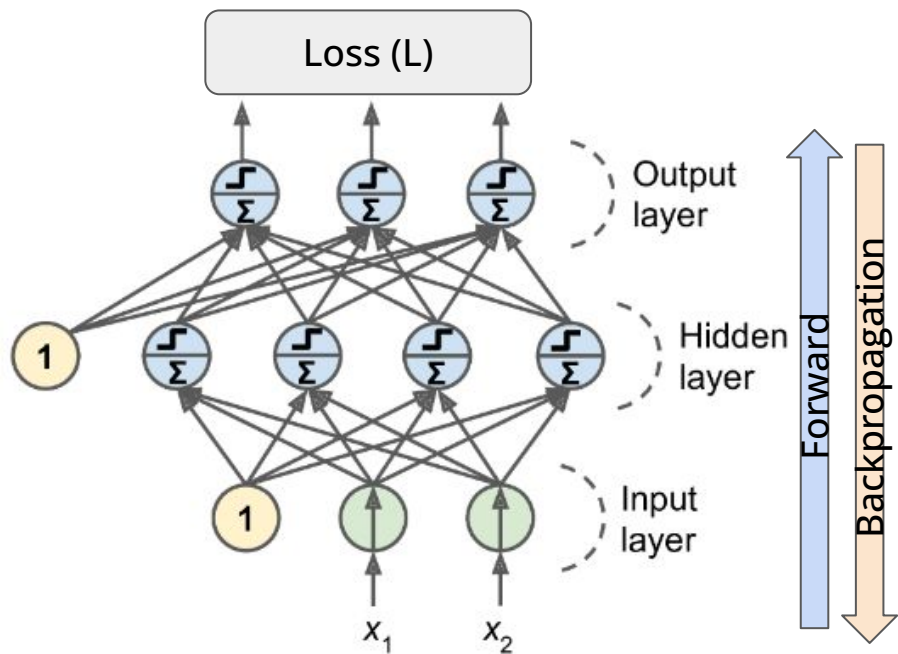
Memoria computacional

Tiempo de entrenamiento

Riesgo de sobreajuste



Redes Neuronales: Perceptron multicapa.



$$\theta = ((W^h, b^h), (W^o, b^o))$$

Entrenamiento: determinar los valores de los parámetros que minimicen la función de Loss.

$$\hat{\theta} = \arg \min_{\theta} \sum_i^{N_{train}} L(f_{\theta}(x_i), y_i)$$

Descenso por gradiente (estocástico)

Forward: cálculo de L para un lote.

$$L(\theta_t) = \sum_j^{n_{lote}} L(x_i, y_i, \theta_t) + \lambda R(\theta_t)$$

Backpropagation: cálculo del gradiente.

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

Redes Neuronales

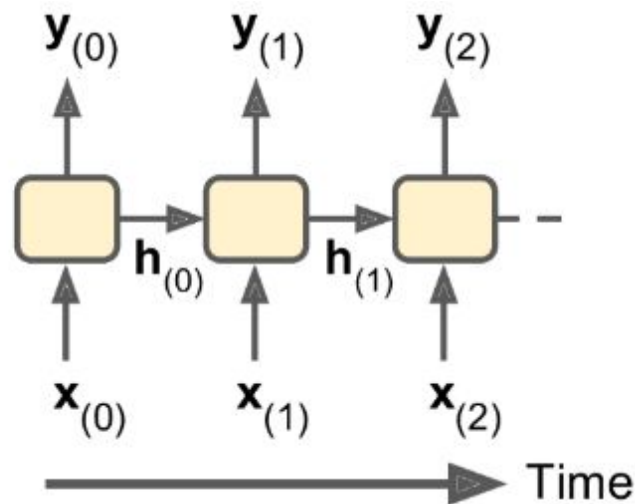
Hiperparámetros:

- **# de capas:** determina la profundidad de la red.
- **# de unidades por capa:** controla la capacidad de representación del modelo.
- **Bias:** término independiente que desplaza la activación de cada neurona.
- **Función de activación:** introduce no linealidad (ReLU, sigmoid, tanh, etc.).
- **Tasa de aprendizaje (learning rate, η):** tamaño de los pasos durante la optimización.
- **Tamaño del lote (batch size):** cantidad de ejemplos utilizados para calcular cada actualización de los pesos.
- **Coeficiente de regularización (λ):** penaliza modelos demasiado complejos para reducir el sobreajuste.
- **Número de épocas (epochs):** cantidad de veces que el modelo recorre todo el conjunto de entrenamiento.
- **Optimizador:** algoritmo utilizado para actualizar los pesos (SGD, Adam, RMSProp, etc.).
- **Inicialización de pesos:** define los valores iniciales de los parámetros (Xavier, He, etc.), influyendo en la velocidad y estabilidad del entrenamiento.
- **Dropout:** porcentaje de neuronas desactivadas durante el entrenamiento para mejorar la generalización.
- **Función de pérdida (loss function):** criterio que el entrenamiento intenta minimizar (MSE, entropía cruzada, etc.).

Redes recurrentes (RNN)

Redes recurrentes (RNN)

- Arquitectura específica para datos secuenciales.
- Mismas matrices de pesos para cada instante de la entrada.
 - # parámetros es invariable al largo de secuencia.
 - Reducción de parámetros.
 - Se puede entrenar con datos de largo variable.

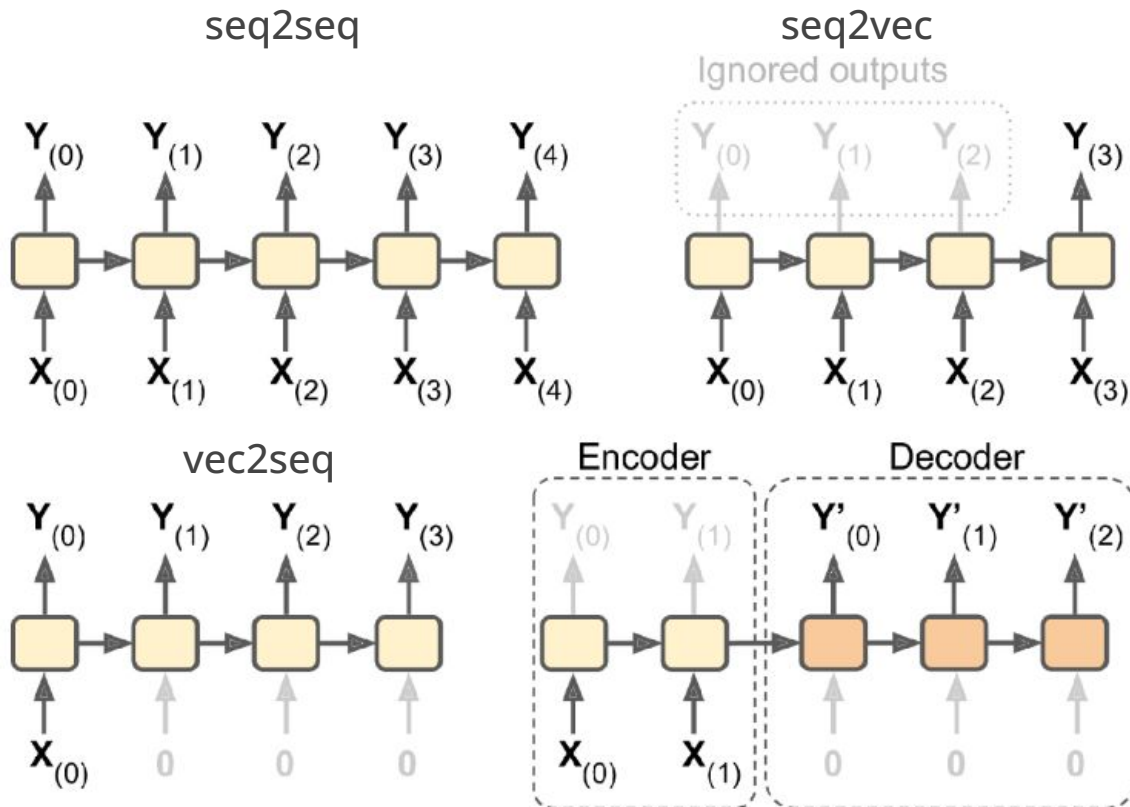


$$h_{(i)} = f\left(x_{(i)} W_{xh} + h_{(i-1)} W_{hh} + b_h\right) \longrightarrow \text{Estado oculto: preserva la información histórica.}$$

$$y_{(i)} = h_{(i)} W_{hy} + b_y \longrightarrow \text{Salida.}$$

Redes recurrentes (RNN)

Diferentes configuraciones



Ejemplos:

- seq2seq: predicción de una semana.
- seq2vec: predicción del siguiente valor.
- vec2seq: descripción de imágenes.
- encoder-decoder: traducción de texto.

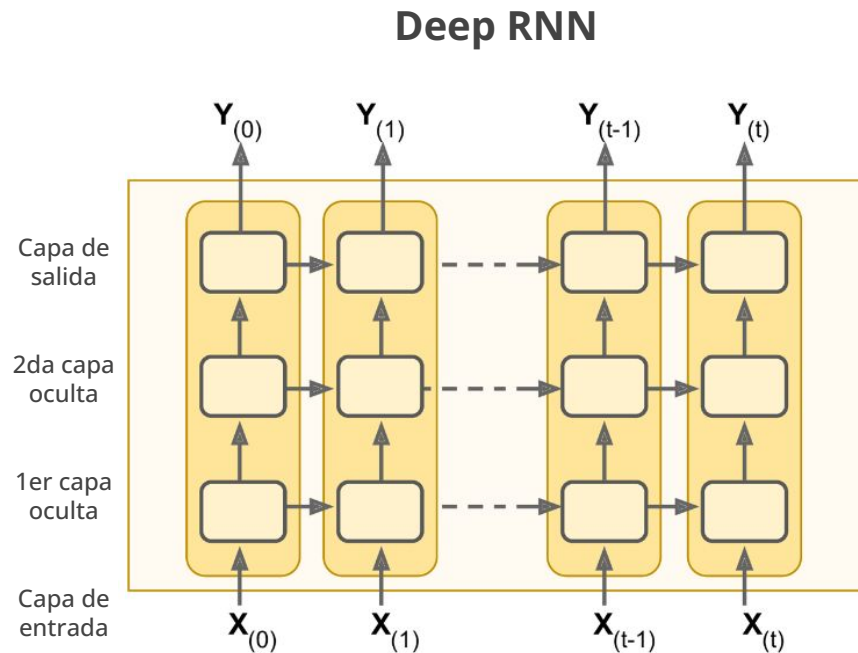
Redes recurrentes (RNN)

Ventajas:

- Reducción de cantidad de parámetros.
- La arquitectura es independiente al largo de secuencia.
- Se pueden procesar secuencias largas sin comprometer la memoria del sistema.

Limitaciones:

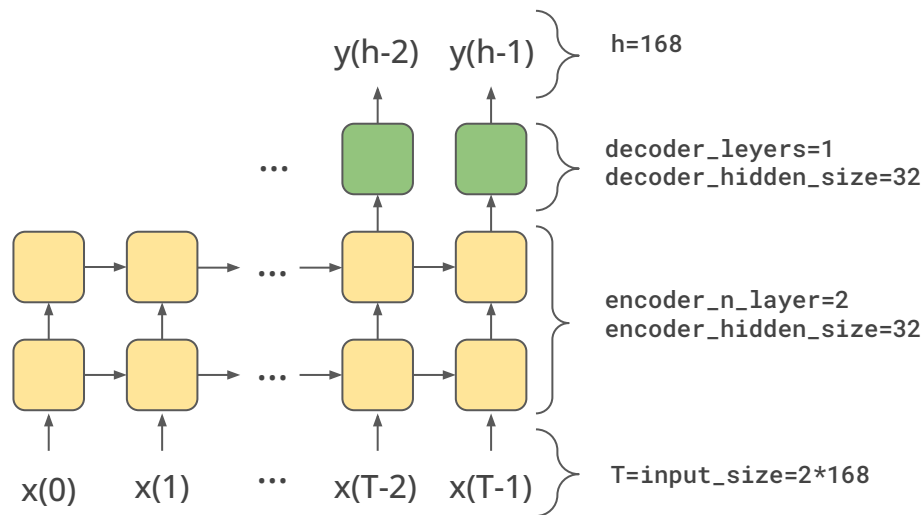
- El entrenamiento puede ser lento. Procesar cada instante depende del instante anterior.
- Desvanecimiento o explosión del gradiente.
- Memoria a corto plazo. Para este problema se utilizan capas/celdas con memoria a largo plazo.
 - Long Short-Term Memory (**LSTM**).
 - Gate Recurrent Unit (**GRU**).



Predicción de más de un valor (1 semana): RNN

Modelo: seq2seq

```
fcst = NeuralForecast(  
    models=[RNN(  
        h=168,  
        input_size=2*168,  
        scaler_type='standard',  
        loss=MSE(),  
        valid_loss=MSE(),  
        encoder_n_layers=2,  
        encoder_hidden_size=32,  
        decoder_layers=1,  
        decoder_hidden_size=32,  
        max_steps=500)])
```

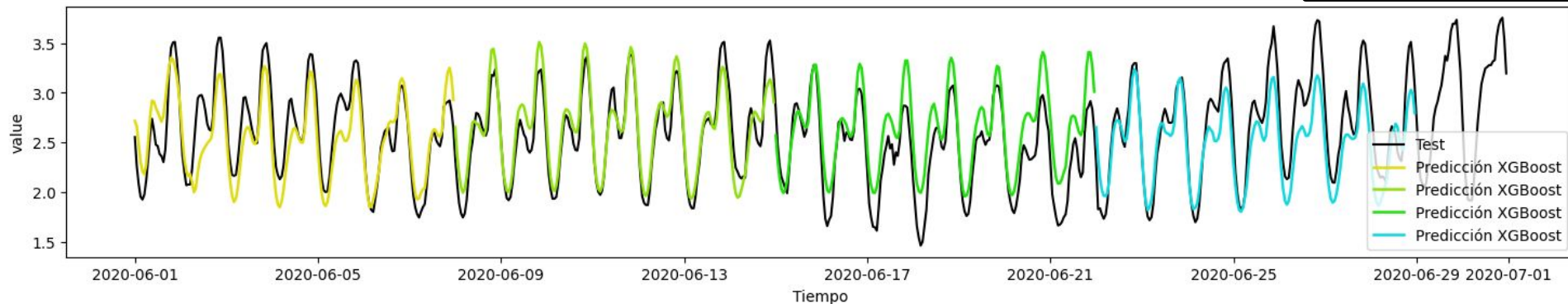


Predicción RNN 1w

XGBoost

MAE = 0.230
MSE = 0.084
MAPE = 0.087

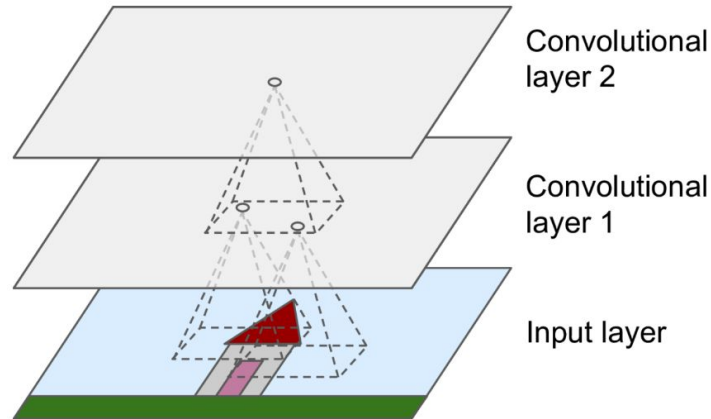
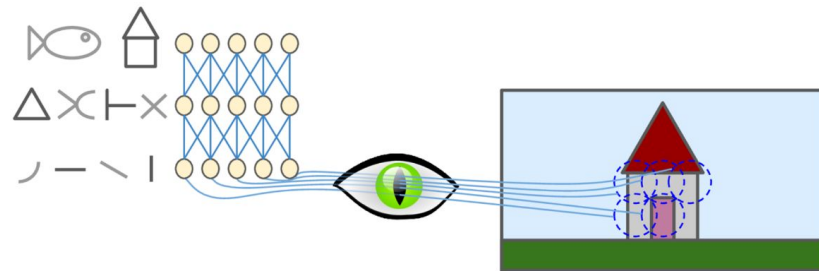
MAE = 0.209
MSE = 0.065
MAPE = 0.083



Redes Convolucionales (CNN)

Redes convolucionales (CNN)

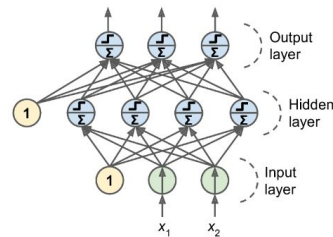
- Inspiradas en el funcionamiento de la corteza visual en mamíferos.
- Revolucionaron el campo de visión artificial.
- Originalmente utilizada para tareas con imágenes:
 - Neuronas: filtros rectangulares muy pequeños.
 - Cada convolución devuelve una salida de dos dimensiones (feature map) que será la entrada de los filtros de la capa siguiente.
 - Las primeras capas capturan información local, mientras que las últimas información regional.



Redes convolucionales (CNN)

Ventajas respecto a MLP:

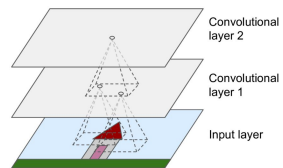
- Reduce drásticamente la cantidad de parámetros.
 - Menos memoria.
 - Menor posibilidad de sobreajuste.
- Aprovecha la correlación espacial de las imágenes.
- Invariante a traslaciones.



VS.

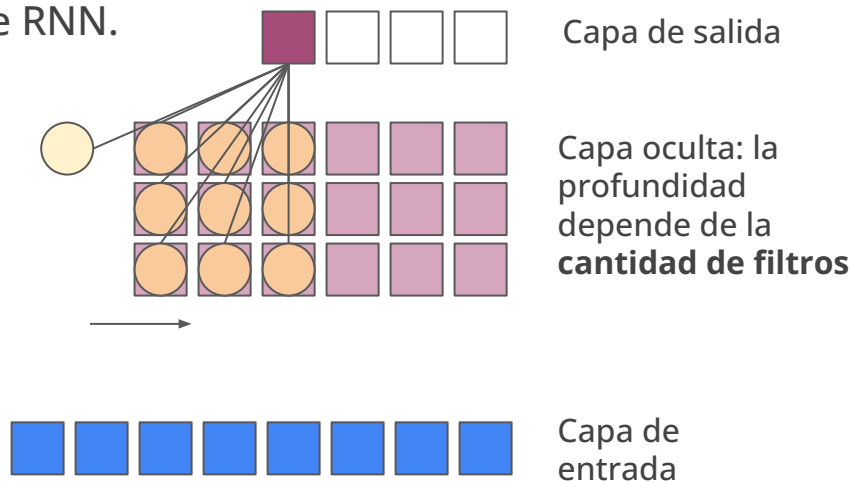
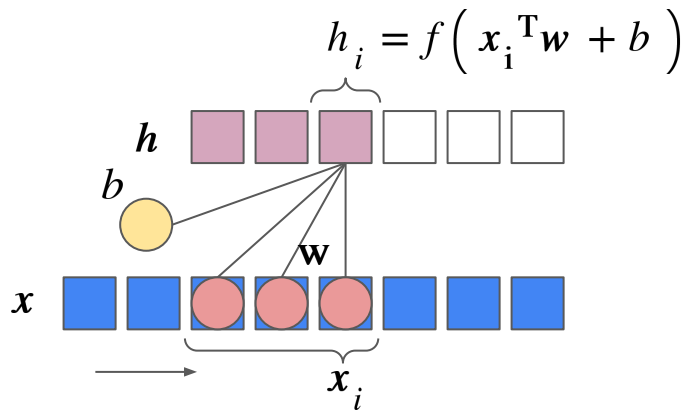
Limitaciones:

- Diseño de capas más complejo. Más hiperparámetros.
- Realiza más operaciones. Costo computacional.



Redes convolucionales unidimensionales (1D CNN)

- Específicas para datos secuenciales: texto, audio, **series temporales**.
- La convolución es en una sola dimensión. El tamaño de los filtros se define solo en esa dimensión.
- Cada convolución devuelve una secuencia (feature map).
- Puede procesar secuencias de largo variable.
- Cálculos paralelizables. Son más rápidas que RNN.

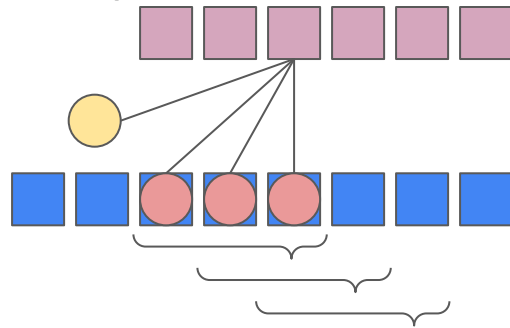


Redes convolucionales unidimensionales (1D CNN)

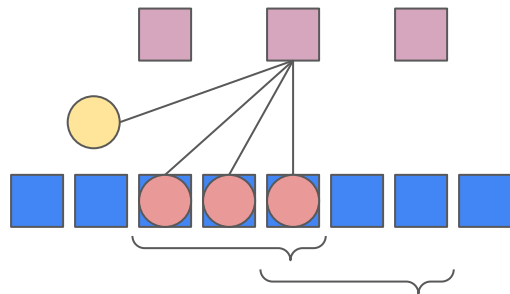
Hiperparámetros:

- # filtros por capa.
- Tamaño del filtro (kernel) en la dirección de convolución.
- Bias.
- Función de activación.
- Stride

stride=1
(por defecto)



stride=2



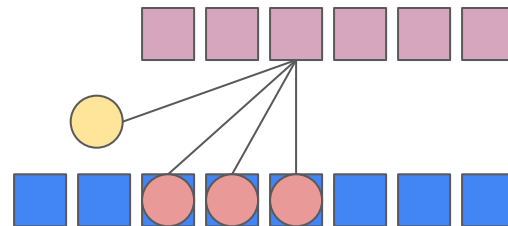
Redes convolucionales unidimensionales (1D CNN)

Hiperparámetros:

- # filtros por capa.
- Tamaño del filtro en la dirección de convolución.
- Bias.
- Función de activación.
- Stride
- Padding

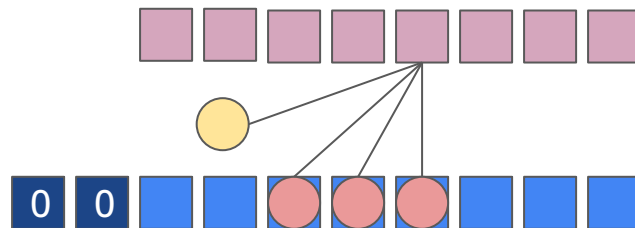
Padding=valid (sin)

Stride=1



Padding=same (con)

Stride=1



Redes convolucionales unidimensionales (1D CNN)

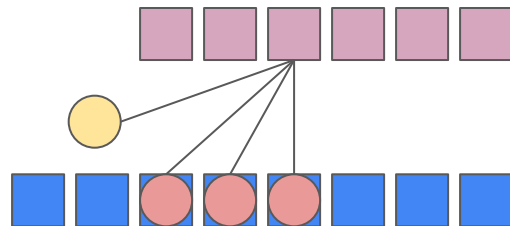
Hiperparámetros:

- # filtros por capa.
- Tamaño del filtro en la dirección de convolución.
- Bias.
- Función de activación.
- Stride
- Padding
- Dilatación

Dilatación=1 (sin)

Stride=1

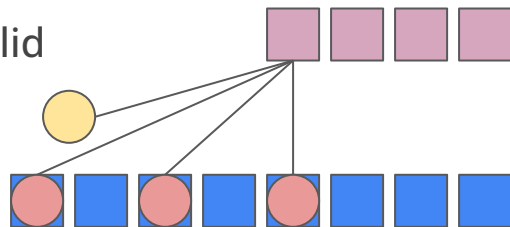
Padding=valid



Dilatación=2 (con)

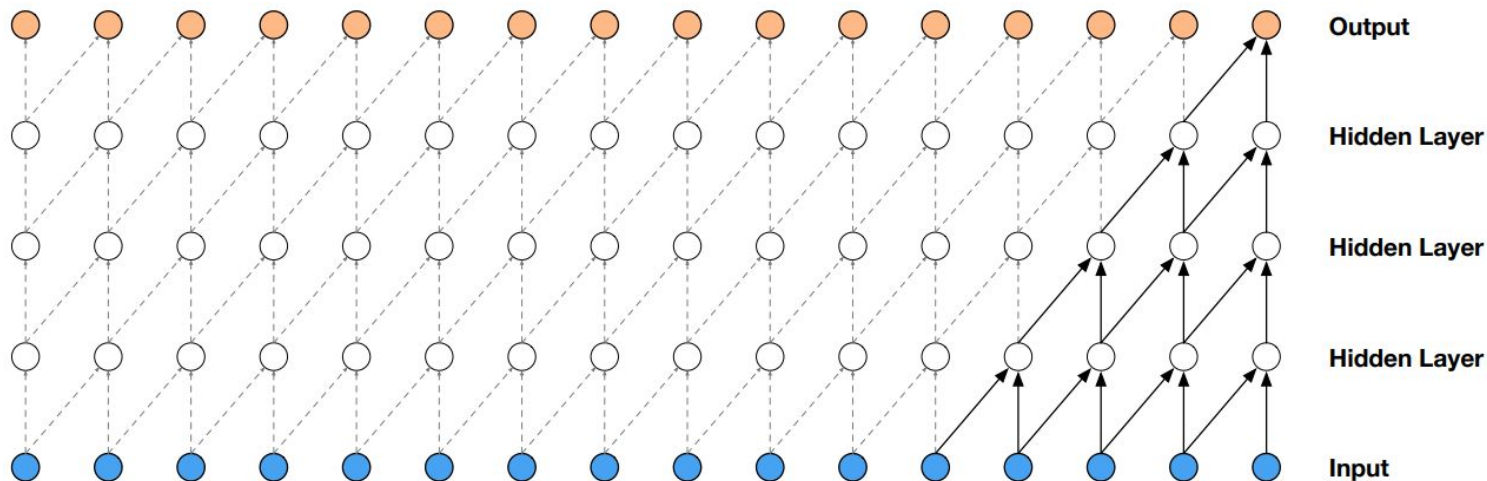
Stride=1

Padding=valid



Redes convolucionales unidimensionales (1D CNN)

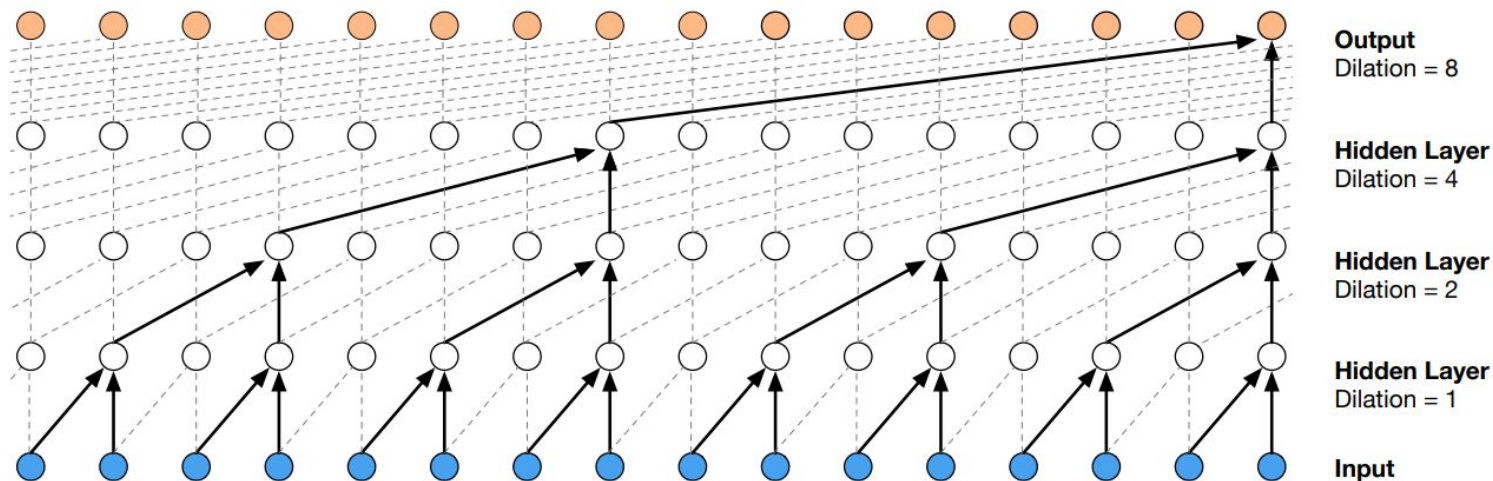
Problemas con las CNN: para que las capas más abstractas "vean" toda la entrada, con filtros pequeños, se necesitan muchas capas.



Redes convolucionales unidimensionales (1D CNN)

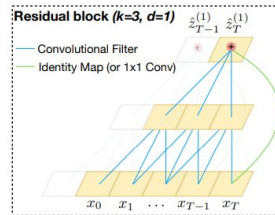
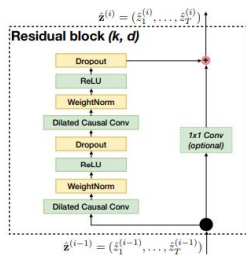
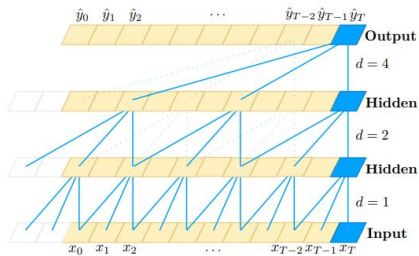
Problemas con las CNN: para que las capas más abstractas "vean" toda la entrada, con filtros pequeños, se necesitan muchas capas.

- Solución: dilataciones aumentadas exponencialmente.



Predicción de más de un valor (1 semana): TCN

```
fcst = NeuralForecast(  
    models=[  
        TCN(h=168,  
            input_size=2*168,  
            loss=MSE(),  
            valid_loss=MSE(),  
            kernel_size=2,  
            dilations=[1,2,4,8,16],  
            encoder_hidden_size=32,  
            decoder_hidden_size=32,  
            decoder_layers=1,  
            max_steps=500,  
            scaler_type='standard'))]
```

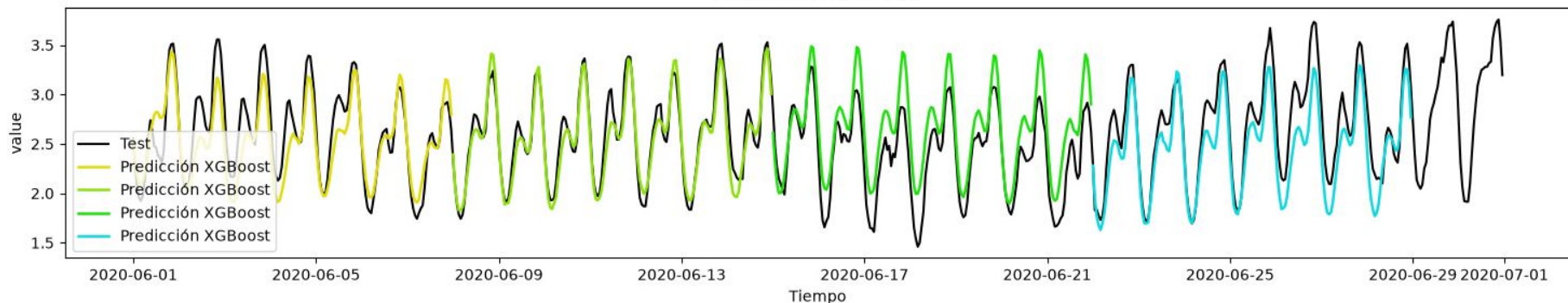


**2 veces más rápido
que RNN.**

RNN

MAE = 0.209
MSE = 0.065
MAPE = 0.083

MAE = 0.197
MSE = 0.060
MAPE = 0.078



Bai, Shaojie, J. Zico Kolter, and Vladlen Koltun. "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling." arXiv preprint arXiv:1803.01271 (2018).

Resumen



FACULTAD DE
INGENIERÍA



UNIVERSIDAD
DE LA REPÚBLICA
URUGUAY

Resumen

Método	Ventajas	Limitaciones
ARIMA / SARIMA	Simple, interpretable, muy buenos para tendencias y estacionalidades.	Solo modelan relaciones lineales.
Árboles / Random Forest / XGBoost	Incorporan fácilmente variables exógenas y features diseñadas manualmente.	Requieren ingeniería de características (lags, rolling, calendario).
MLP	Aprende relaciones no lineales sin diseñar reglas manualmente.	No aprovecha la estructura temporal de la secuencia.
RNN (LSTM, GRU)	Modelan dependencias temporales y secuencias de longitud variable.	Entrenamiento secuencial, más lento y difícil de paralelizar.
CNN (TCN)	Capturan patrones locales, menos parámetros, entrenamiento paralelo y rápido.	Requieren aumentar el campo receptivo (dilataciones) para dependencias largas.

Ideas claves

- No existe un modelo universalmente mejor.
- La elección depende de:
 - cantidad de datos disponibles,
 - horizonte de predicción
 - complejidad de los patrones,
 - necesidad de interpretabilidad,
 - recursos computacionales.
- En problemas reales es recomendable comparar múltiples modelos utilizando un conjunto de validación y métricas apropiadas.

Métricas



FACULTAD DE
INGENIERÍA



UNIVERSIDAD
DE LA REPÚBLICA
URUGUAY

Métricas

Mean absolute error (MAE)

$$\frac{1}{n} \sum_{t=1}^n |y_t - \hat{y}_t|$$

Mean squared error (MSE)

$$\frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2$$

Root mean squared error (RMSE)

$$\sqrt{\frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2}$$

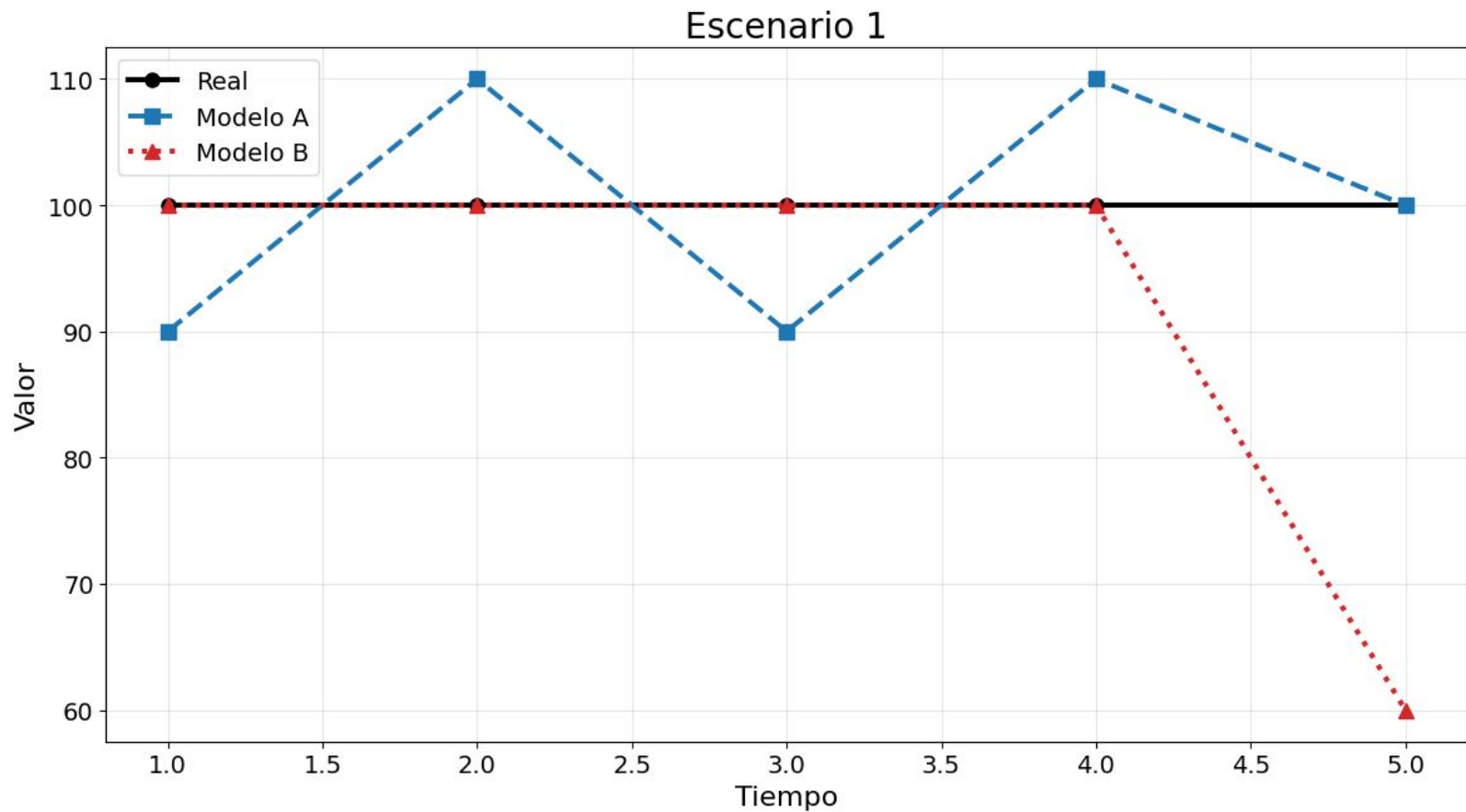
Mean absolute percentage error
(MAPE)

$$\frac{1}{n} \sum_{t=1}^n \left| \frac{y_t - \hat{y}_t}{y_t} \right| \times 100$$

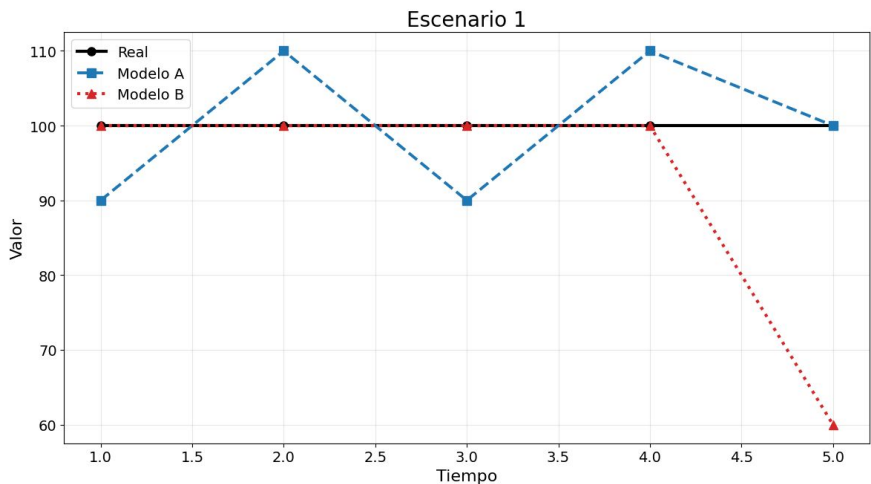
Mean absolute scaled error (MASE)

$$\frac{\frac{1}{n} \sum_{t=1}^n |y_t - \hat{y}_t|}{\frac{1}{n-1} \sum_{t=2}^n |y_t - y_{t-1}|}$$

Métricas: ejemplos



Métricas: ejemplos



Real	100	100	100	100	100
Mod A	90	110	90	110	100
Mod B	100	100	100	100	60
Tiempo	1	2	3	4	5

$$\frac{1}{n} \sum_{t=1}^n |y_t - \hat{y}_t|$$

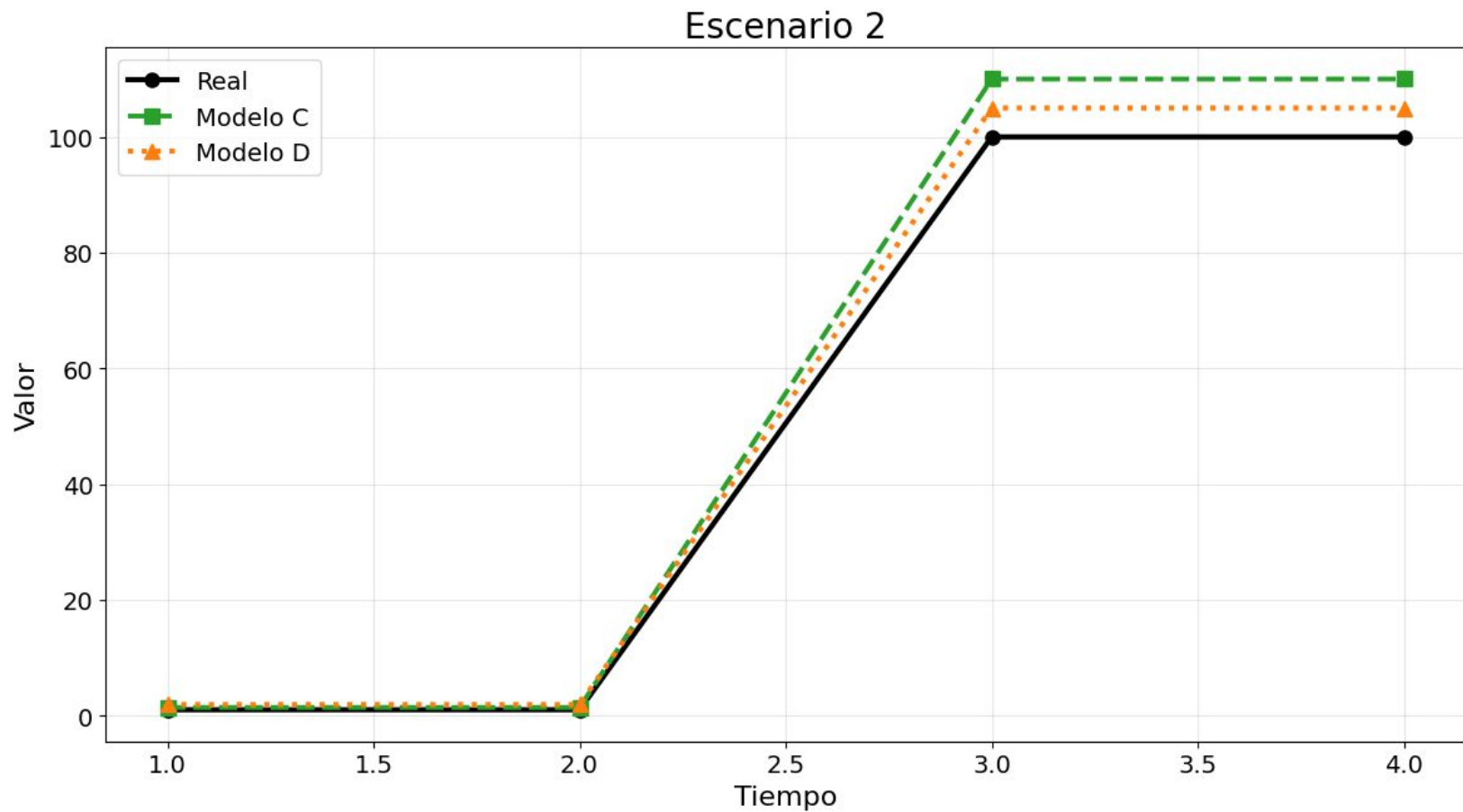
$$\frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2$$

$$\sqrt{\frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2}$$

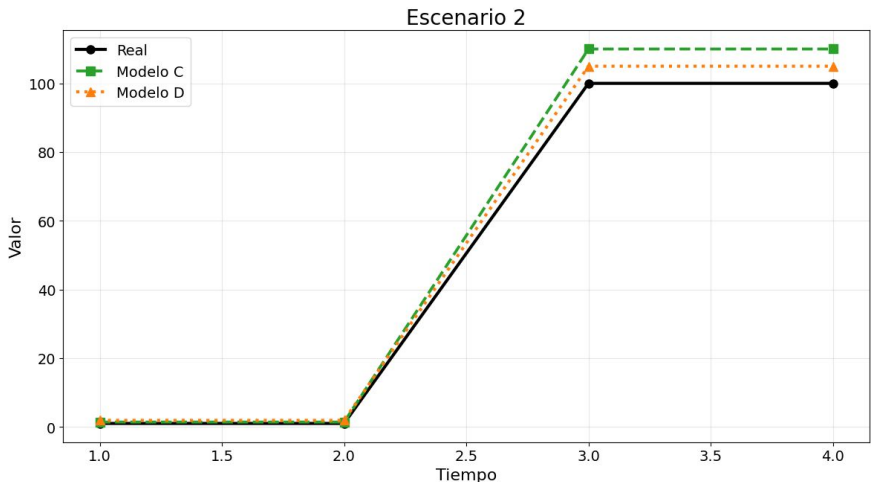
$$\frac{1}{n} \sum_{t=1}^n \left| \frac{y_t - \hat{y}_t}{y_t} \right| \times 100$$

	Mod A	Mod B
MAE	8	8
MSE	80	320
RMSE	8.9	17.9
MAPE	8	8

Métricas: ejemplos



Métricas: ejemplos



Real	1.0	1.0	100	100
Mod C	1.4	1.4	110	110
Mod D	1.9	1.9	105	105
Tiempo	1	2	3	4

$$\frac{1}{n} \sum_{t=1}^n |y_t - \hat{y}_t|$$

$$\frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2$$

$$\sqrt{\frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2}$$

$$\frac{1}{n} \sum_{t=1}^n \left| \frac{y_t - \hat{y}_t}{y_t} \right| \times 100$$

	Mod C	Mod D
MAE	5.2	3.0
MSE	50	13
RMSE	7	4
MAPE	25	47